Popravni test iz predmeta
Algoritama i struktura podataka

Drugi praktični test održan 10.01.2024.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je closed book test, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
4. Vodite računa da imate sve spremno što vam je neophodno.
5. Isključite mobilne telefone i odložite ih na katedru.
6. Ukupan broj bodova na testu je 30 bodova.
7. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
8. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
9. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_Test2_GrupaE_IT1g2021**
10. **Minimalni broj bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka je 5.**
11. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime i prezima i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Završavanje implementacije BFS algoritma*

Osnovna metoda BFS algoritma nije potpuna. Dopuniti implementaciju ove metode koja treba da omogući kompletno izvršavanje ovog algoritma.

a) Napisati algoritam.
b) U *Main* metodi kreirati proizvoljan graf. Obezbediti način za testiranje i proveru nove funkcionalnosti, odnosno poziv ***BFS*** metode.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***public void BFS(int startVertex).***

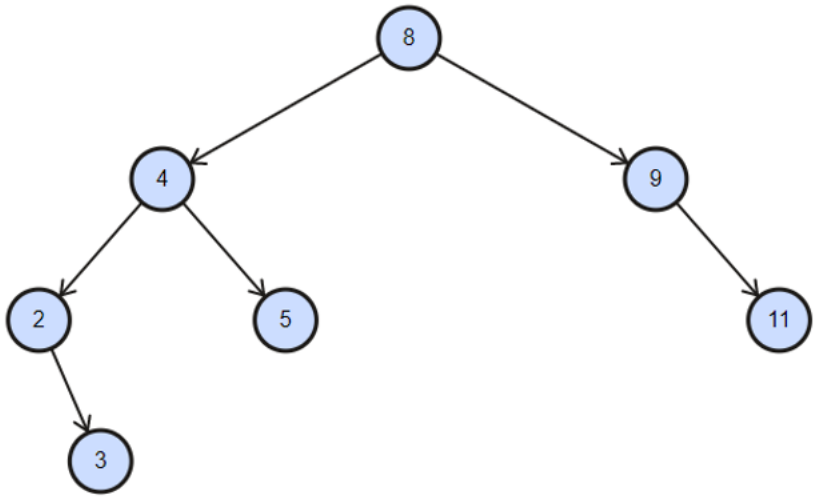
Zadatak 2 - Prebrojavanje čvorova koji nisu na poziciji listova

U okviru standardne strukture klase ***BST*** napisati implementaciju metode koja će omogućiti ispisivanje svih čvorova koji se nalaze na poziciji listova.

Dodatne napomene za rešavanje zadatka:

- U okviru klase ***BST*** kreirati javnu metodu sa potpisom ***public void countNonLeaves()***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private Node countNonLeavesBST(Node curr)***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se omogućiti prolazak kroz stablo od korena ka listovima i njihovo prebrojavanje.
- Listom se klasifikuje svaki čvor koji nema nijedno dete.
- Može se kreirati pomoćna promenljiva na nivou klase BST koja će pomoći u prebrojavanju.
- Algoritam **OBAVEZNO** pisati u okviru klase ***BST***

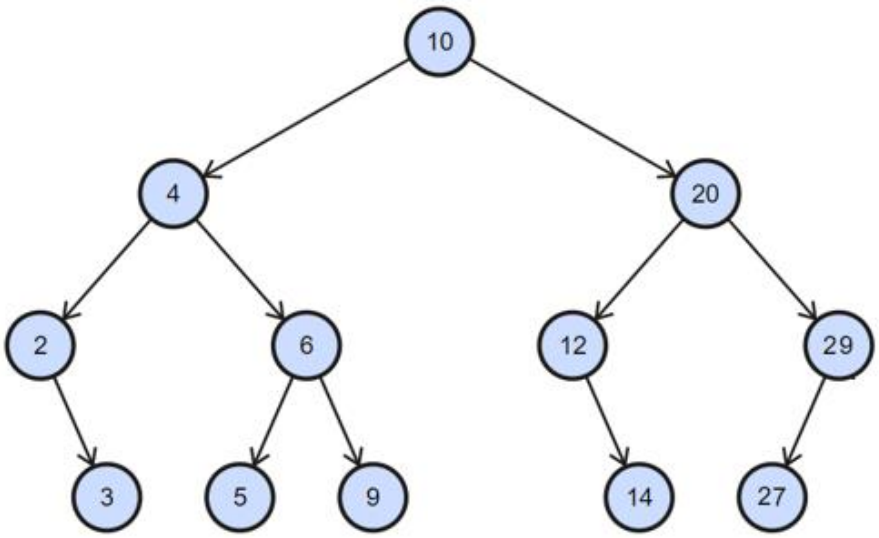
Primeri:



Slika 1 - Primer BST-a

Input 1: *BST sa slike 1*

Output 1: *4*



Slika 2 - Primer BST-a

Input 2: *BST sa slike 2*

Output 2: *7*

Zadatak 3 - *Estimirana sličnost sekvenci nakon određene podsekvence*

Napisati algoritam koji računa **estimiranu** sličnost sekvenci nakon određene podsekvence. Algoritam kao parametre dobija sekvence i subsekvencu koja označava početak subsekvenci čiju sličnost treba estimirati. Algoritam implementirati uz pretpostavku da data subsekventa (npr. MKC iz 1. primera) sigurno postoji u sekvencama koje su date i pojavljuje se samo jednom u njima.

a) Napisati algoritam.
b) U *Main* metodi obezbediti testiranje, pozivom metode sa parametrima koji su dati u primerima.

Dodatne napomene za rešavanje zadatka:

- Koristiti *search* metodu *KarpRabinAlgorithm* klase i klase *MinHash* i *Jaccard,* koje su date u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static double percentOfSimilarityAfterSubsequence(string str1, string str2, string substr).***
- Prilikom kreiranja MinHash objekta, za vrednosti parametara ***universeSize*** i ***numHashFunctions*** prosleđivati vrednosti ***12*** i ***4***
- Parametar ***q*** funkcije *search* postaviti na ***substr.Length+1***
- **Dozvoljeno** je izmeniti povratan tip metode *search* (npr. umesto *void* imati *List<int>*)

Napomene koje mogu biti od pomoći prilikom rešavanja zadatka:

- Kako biste string konvertovali u niz karaktera sa kojim se može vršiti manipulacija, koristiti metodu ***ToCharArray()*** po principu: ***str.ToCharArray()***
- Kako biste od niza karaktera dobili string, koristiti ***new string(char[] arrayOfChars)*** po principu: ***new string(charArray)***
- Karakteri se ne moraju eksplicitno konvertovati u celobrojni tip prilikom ubacivanja u *List<uint>* strukturu

Primer:

- Input: “ACCCBBW**MKC**CCBCAC”, “HRCC**MKC**AKYCRBHVB”, “**MKC**”
- Output: 0,2

(Estimirana sličnost subsekvenci CCBCAC i AKYCRBHVB je 0,2)